

Character Set, ASCII & Unicode

Java Programming • ICSE Class 9

NeverStop • By Sourabh Gorai



Character Set

The alphabet of computers



ASCII

Numbers as characters



Unicode

One code for all scripts

■ What You'll Learn

- What is a Character Set?
- How ASCII encodes 128 symbols
- Unicode — the global solution
- Java's char data type & escape sequences
- ASCII tricks in Java programs



◆ **Covers:** Character Set | ASCII Table (0–127) | Unicode | Java char | Memory Tricks | Practice Q's

Prepared as comprehensive study notes for first-time programmers.

■ Definition

A Character Set is a collection of characters (letters, digits, symbols, and special marks) that a computer understands. Just like how we have an alphabet in English (A–Z), a computer has its own defined set of characters it can store and process.

■ Real-Life Analogy

Think of a Character Set like a stamp kit. Each stamp = one character.
The kit only contains the stamps it was made with — nothing more!

■ Types of Characters in a Character Set

Aa

Letters

Uppercase A–Z and
Lowercase a–z

09

Digits

Numbers 0 through 9

—

Special

+ - * / @ # \$ % & () etc.



Whitespace

Space, Tab (t), Newline (n)

Esc

Control

Non-printable: backspace,
delete, bell...

■ Why Does a Computer Need a Character Set?

Computers only understand 0s and 1s (binary numbers).

Every character must be mapped to a unique number so the CPU can store it.

When you press 'A' on the keyboard, the computer stores a number, not the letter itself.

The Character Set is the rule-book that says: 'A = number 65'.

■ Popular Character Sets — A Brief Timeline



■ ASCII — Full Form & Definition

ASCII = American Standard Code for Information Interchange

Developed in 1963 by the American Standards Association (ASA).
It assigns a unique number (0–127) to 128 characters.

128

Total
Characters

7

Bits used
($2^7=128$)

0-31

Control
Chars

32

Space
(first printable)

65

'A' (uppercase
A starts here)

97

'a' (lowercase
a starts here)

■ ASCII Groups at a Glance

0–31**range**

Control Characters

Non-printable. They control devices.

Examples: 0=Null, 8=Backspace, 9=Tab, 10=Newline, 13=Enter

32–64**range**

Printable Symbols & Digits

Space(32), !(33), "(34), #(35)... 0(48) to 9(57)

Also: @(64), brackets, punctuation

65–90**range**

Uppercase Letters (A–Z)

A=65, B=66, C=67 ... Z=90

Tip: Each letter = previous letter + 1

97–122**range**

Lowercase Letters (a–z)

a=97, b=98, c=99 ... z=122

Tip: lowercase = uppercase + 32

■ Must-Know ASCII Values

Character	ASCII Value	In Java
'A'	65	char c = 'A'; → stores 65
'Z'	90	char c = 'Z'; → stores 90
'a'	97	char c = 'a'; → stores 97
'z'	122	char c = 'z'; → stores 122
'0'	48	char c = '0'; → stores 48 ← NOTE: '0' ≠ 0
'9'	57	char c = '9'; → stores 57
' '	32	Space character — first printable ASCII
'\n'	10	Newline — moves cursor to next line
'\t'	9	Tab — moves cursor by a tab stop

ASCII Quick-Reference Table (32 – 127)

Printable Characters Only

■ Digits 0–9
 ■ Uppercase A–Z
 ■ Lowercase a–z
 ■ Symbols

■	!	"	#	\$	%	&	'
32	33	34	35	36	37	38	39
(	)	*	+	,	-	.	/
40	41	42	43	44	45	46	47
0	1	2	3	4	5	6	7
48	49	50	51	52	53	54	55
8	9	:	;	<	=	>	?
56	57	58	59	60	61	62	63
@	A	B	C	D	E	F	G
64	65	66	67	68	69	70	71
H	I	J	K	L	M	N	O
72	73	74	75	76	77	78	79
P	Q	R	S	T	U	V	W
80	81	82	83	84	85	86	87
X	Y	Z	[	\	]	^	_
88	89	90	91	92	93	94	95
`	a	b	c	d	e	f	g
96	97	98	99	100	101	102	103
h	i	j	k	l	m	n	o
104	105	106	107	108	109	110	111
p	q	r	s	t	u	v	w
112	113	114	115	116	117	118	119
x	y	z	{		}	~	■

■ **Java Tip:** `System.out.println((int)'A'); // 65`

■ Why Was ASCII Not Enough?

- ! ASCII was designed in the USA, so it mostly covers English letters and common symbols.
- ! Languages like Hindi (■■■■■■■■■■), Chinese (■■■), Arabic (■■■■■), and Japanese (■■■■) had no place in ASCII.
- ! Each country created its own character set, which caused big compatibility problems.
- ! A file written in Germany could not be read correctly in Japan.
- ! The internet needed one universal standard, and that is where Unicode came in.

■ What is Unicode?

Unicode is a universal character encoding standard that assigns a unique number (called a Code Point) to every character of every language in the world.

Unicode can represent 1,114,112 characters — over 1 million!

■ Unicode Key Facts

Code Point	A unique number for each character. Example: U+0041 = 'A'
Range	U+0000 to U+10FFFF — about 1.1 million possible values
First 128	U+0000 to U+007F matches ASCII exactly
Planes	Unicode has 17 planes; Plane 0 is the Basic Multilingual Plane
Emoji	Emojis are Unicode characters too, such as U+1F60A ■

■ Unicode Encodings (How Code Points are Stored)

UTF-8	UTF-16	UTF-32
Variable: 1–4 bytes Most popular. Used by all websites. Backward compatible with ASCII.	Variable: 2–4 bytes Used internally by Java! Each char is 2 bytes (16 bits).	Fixed: 4 bytes Simple but wastes memory. Each code point = exactly 4 bytes.

■ **Java Note:** Java stores characters using UTF-16 encoding. The 'char' data type is 2 bytes = 16 bits.

■ The 'char' Data Type in Java

Size: 2 bytes (16 bits) — one UTF-16 character
Range: 0 to 65,535 — enough for the Basic Multilingual Plane
Syntax: char variableName = 'A'; // single quotes only
Default: Default value: '\u0000' (null character)
Numeric: char is numeric: 65 becomes 'A'

■ Java Code Examples — char in Action

Declaring a char

```
char letter = 'A';
char digit = '5';
char grade = 'B';
System.out.println(letter); // Output: A
```

char from ASCII value

```
char c = 65; // Same as 'A'
System.out.println(c); // A
System.out.println((int)c); // 65
System.out.println(c + 1); // 66 (int result!)
```

Arithmetic with char

```
char upper = 'A';
char lower = (char)(upper + 32);
System.out.println(lower); // a
// 'A'(65) + 32 = 97 = 'a'
```

Unicode escape

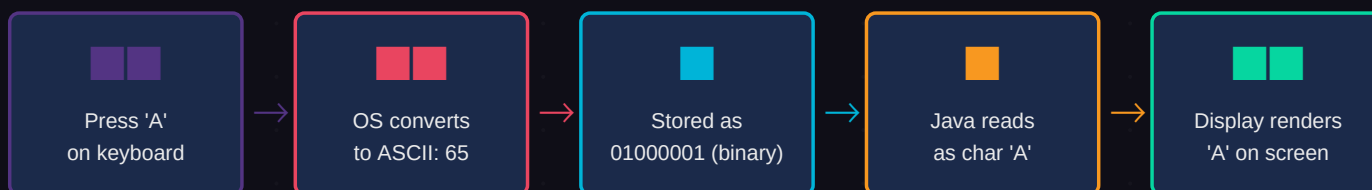
```
char heart = '\u2665'; // ♥
char smiley = '\u263A'; // ☺
System.out.println(heart); // ♥
System.out.println(smiley); // ☺
```

■■ Escape Sequences in Java

\n	10	Newline — moves to next line	\t	9	Tab — horizontal tab space
\r	13	Carriage Return — moves to line start	\'	39	Single Quote — needed inside char literal
\"	34	Double Quote — needed inside String	\\	92	Backslash — literal backslash character
\0	0	Null — null character (end of string in C)	\u	—	Unicode escape — \uXXXX for any Unicode char

ASCII		Unicode
1963	Year	1991
128 only	Characters	1,114,112+
7 bits	Bits per char	Variable (UTF-8/16/32)
English only	Languages	All world languages
—	Backward compat.	U+0000–U+007F = ASCII
Very limited	Symbol support	Math, emojis, scripts, all!
Partially (char was ASCII)	Used by Java	Yes — Java uses UTF-16
Outdated	Web standard	UTF-8 is THE web standard

■ ■ How 'A' is Stored in Memory (Visualization)



■ Interesting Java + ASCII/Unicode Facts

char + char = int!	In Java: 'A' + 'B' = 65 + 66 = 131 (an int, not a char!)
char to int cast	(int)'A' gives 65. Casting char to int reveals its ASCII/Unicode value.
int to char cast	(char)65 gives 'A'. You can convert any Unicode code point to a char.
String of one char	String.valueOf('A') or Character.toString('A') converts char to String.
isLetter / isDigit	Character.isLetter('A') → true; Character.isDigit('5') → true
toUpperCase / toLowerCase	Character.toUpperCase('a') → 'A'; Character.toLowerCase('Z') → 'z'

■ Super Memory Tricks

TRICK 1

'A' = 65 → Remember: A is the 1st letter. 65 starts the uppercase group.
Say aloud: 'A is 65, B is 66, C is 67' — after three, you'll never forget!

TRICK 2

Lowercase = Uppercase + 32 → 'a' = 'A' + 32 = 65 + 32 = 97

Memory hook: 'LOWER is LARGER by 32' — lowercase letters have bigger ASCII values.

TRICK 3

Digits start at 48 → '0' = 48, '1' = 49, '9' = 57

Remember: '0' + 48 trick — 'Zero is NOT zero, it's 48!' This trips up many students.

TRICK 4

Space = 32 → The first VISIBLE ASCII character is Space (32)

Rhyme: 'Thirty-two, what do I see? A space — the gap between you and me!'

TRICK 5

ASCII = 7 bits → $2^7 = 128$ characters (0 to 127)

Fist method: hold up 7 fingers and say 'seven bits = one-twenty-eight!'

TRICK 6

Unicode uses U+XXXX format → U+0041 = 'A' (0041 in hex = 65 in decimal)
U+ means Unicode. The number after is in hexadecimal!

■ Practice Questions

Q1. What is a Character Set? Why do computers need it? Theory (2 marks)

Q2. What does ASCII stand for? How many characters can it represent? Theory (2 marks)

Q3. What is the ASCII value of 'D'? (Hint: 'A' = 65) Calculate (1 mark) → Answer: 68

Q4. What is the output?
char c = 'a';
System.out.println((int)c); Java Output (2 marks) → Answer: 97

Q5. Write a Java program to print the ASCII values of A to E. Program (4 marks)

Q6. What is Unicode? How is it different from ASCII? Theory/Compare (3 marks)

Q7. What is the output of: System.out.println('A' + 1); Explain why. Tricky Java (2 marks) → Answer: 66 (int, not char!)

Q8. What is the ASCII value of the SPACE character? Why is it important? Theory (1 mark) → Answer: 32

■ You've Got This! Every expert was once a beginner. Keep coding! ■

Review this sheet before exams and you'll ace the Character Set topic!